

Vision

PREPARED BY Energy Hippo implementation team

Hippo Synthetic Customer is a controlled fake customer that can be loaded into dev and demo environments to prove that ingestion, reports, gateways, and integrations still work end to end.

The project should behave like a small internal product:

- It has named releases of synthetic data.
- It has a source model that explains why the data exists.
- It generates artifacts instead of storing disconnected copies.
- It includes validation checks for expected downstream behavior.

Why This Exists

Hippo has many integrations and data paths. Testing them independently misses failures that only appear when customer data moves across systems. A synthetic customer gives us one stable reference point that can be loaded repeatedly and used for demos, regression testing, and gateway validation.

Core Design

The repository owns a synthetic source database and scenario manifests. Generator code will transform that source into the same kinds of artifacts Hippo already ingests:

- MDEF files.
- BIF files.
- Backfill inputs.
- Utility-provider-like data files.
- Gateway payloads.
- Integration export payloads.

Those outputs should be reproducible. If two developers generate `DemoUniversityV1` from the same commit, they should get the same customer, accounts, meters, bills, readings, and expected assertions.

Data Safety

The customer identity must be fictional. Real public context is allowed where it is useful and safe:

- Public weather stations.
- Public utility territory assumptions.
- Public geography.
- Fictional addresses in real regions.

Private customer data must not be copied into this repository, even if anonymized.

Rollout Plan

1. Define DemoUniversityV1 with a small campus and apartment complex.
2. Create the synthetic source schema.
3. Generate a minimal set of bills and interval data.
4. Produce one or two ingestion artifact types.
5. Load the customer into a dev environment through normal ingestion paths.
6. Add assertions for rollups, reports, idempotency, and integration exports.
7. Expand coverage until every major gateway and integration has at least one scenario.

Quality Bar

A scenario is not complete until it has:

- Source data.
- Generated ingestion artifacts.
- A documented reason for existing.
- Expected outcomes.
- A repeatable validation path.